However, this sort of scaling has a potential risk. If the whole machine crashes in an emergency or due to a hardware failure, data may be lost, because all the nodes hosting the data exist on the same machine. Of course, we have backups, so restoration is possible, but it's painful. Our replicas are hosted on the same machine, albeit different nodes, which is asking for trouble.

3.6.2 Scaling out (horizontal scaling)

Alternatively, we can scale our environment out (horizontally). Instead of attaching additional RAM and memory to the existing machines, we can throw in a number of new machines (probably VMs with less resource power than the fat machines used for vertical scaling) to form a horizontally scaled farm.

These new VMs are booted up as new nodes, thus joining the existing Elasticsearch cluster. As soon as the new nodes join the cluster, Elasticsearch, being a distributed architected engine, instantly distributes data to them. Creating VMs is easy, especially using modern infrastructure as code (IaC) tools like Terraform, Ansible, Chef, and so on, so this approach tends to be favored by many DevOps engineering teams.

Let's wrap up this chapter here. It is your guide to understanding the fundamentals of Elasticsearch: its moving parts, low-level blocks, and search concepts. We talk more about the concepts and fundamentals, along with examples, in the next two chapters, so stay tuned.

Summary

- Elasticsearch expects data to be brought in to be indexed. Data sources can include simple files, databases, live streams, Twitter, and so on.
- In the indexing process, data undergoes a rigorous analysis phase, during which advanced data structures like inverted indexes are created.
- Data is retrieved or searched via the search APIs (along with the document APIs for single document retrieval).
- Incoming data must be wrapped up in a JSON document. Because the JSON document is the fundamental data-holding entity, it is persisted to shards and replicas.
- Shards and replicas are Apache Lucene instances whose responsibility is to persist, retrieve, and distribute documents.
- When we start up the Elasticsearch application, it boots up as a one-node, single-cluster application. Adding nodes expands the cluster, making it a multi-node cluster.
- For faster information retrieval and data persistence, Elasticsearch provides advanced data structures like inverted indexes for structural data (such as textual information) and BKD trees for nonstructural data (such as dates and numbers).
- Relevancy is a positive floating-point score attached to retrieved document results. It defines how well the document matches the search criteria.